Compact Enumeration of Maximal Closed Substrings in Run-Length Encoded Strings

Haruki Umezaki[1], Hiroki Shibata[2], Yuto Nakashima[3], and Shunsuke Inenaga[3]

[1] Department of Information Science and Technology, Kyushu University
umezaki.haruki.314@s.kyushu-u.ac.jp
[2] Joint Graduate School of Mathematics for Innovation, Kyushu University
shibata.hiroki.753@s.kyushu-u.ac.jp
[3] Department of Informatics, Kyushu University
{nakashima.yuto.003, inenaga.shunsuke.380}@m.kyushu-u.ac.jp

Abstract. A string w is *closed* if $|w| = 1$, or if w has a non-empty border occurring only as its prefix and suffix. A *maximal closed substring* (*MCS*) is a maximal occurrence of a closed string; equivalently, it is a *maximal closed repeat* (*MCR*). We study MCS enumeration directly from the *run-length encoding* (*RLE*) of a string. For a string T of length n with RLE size m, we introduce a compact family representation for all MCS occurrences. We prove that $O(m^2)$ families are always sufficient and sometimes necessary. The representation relies on consecutive occurrence pairs of longest borders, classified by the RLE length of the border. The non-unary non-periodic cases are handled uniformly by a sparse suffix tree on run-start suffixes and height-based three-sided range reporting over RLE-boundary point sets; periodic cases are treated separately. Using McCreight's balanced priority search trees, the compact representation $\mathcal{F}$ of all MCSs can be listed in $O(m \log^2 m + |\mathcal{F}| \log m)$ time with $O(m)$ working space.

1 Introduction

Repeated substrings are among the most fundamental objects in string processing. Classical examples include squares, runs, maximal repeats, gapped repeats, and their algorithmic variants. A string is called *closed* if it is of length one, or if it has a non-empty border that occurs in the string only as the prefix and the suffix. Badkobeh et al. [2] introduced *maximal closed substrings* (*MCSs*), namely occurrences of closed substrings that cannot be extended to the left nor to the right into a longer closed substring. They showed that MCSs with exponent at least two are exactly runs, while the other MCSs form a subclass of maximal gapped repeats. They also showed that all MCSs in a string of length n can be computed in $O(n \log n)$ time with $O(n)$ space. Their algorithm also implies an $O(n \log n)$ upper bound for the number of MCSs occurring in any string of length n [2]. Kosolobov [5] later introduced *maximal closed repeats* (*MCRs*) and showed a matching $\Omega(n \log n)$ lower bound on their number in a string of length n.

This paper studies MCSs from the viewpoint of compressed computation. We focus on the most basic compression scheme, *run-length encoding* (RLE). Given an RLE string $T = c_1^{e_1} \cdots c_m^{e_m}$ of size m with $c_i \neq c_{i+1}$ and $e_i \geq 1$, our goal is to describe and enumerate the MCSs of T without expanding T. Since the number of MCSs can be $\Theta(n \log n)$ in the expanded length n [5], it is hopeless to output all occurrences within a time bound depending polynomially only on m in the worst case. We therefore aim at a compact representation by RLE-based families.

Our classification is RLE-based in spirit, as in the RLE-based study of minimal absent words [1], but the structural object is a pair of consecutive occurrences of a longest border: the absence of an intermediate occurrence is what makes the spanned factor closed. This leads to *valid pairs*, our canonical representation of MCS occurrences of length greater than one. The rest of this paper shows the following:

Theorem 1. *Given an RLE of size m representing string T, a family $\mathcal{F}$ of size $O(m^2)$ that represents all MCSs in T can be computed in $O(m \log^2 m + |\mathcal{F}| \log m)$ time with $O(m)$ working space.*

2 Preliminaries

Let Σ be an ordered alphabet. For a string T, $|T|$ denotes its length, and $T[i]$ denotes the ith character of T for $1 \leq i \leq |T|$. For $1 \leq i \leq j \leq |T|$, let $T[i..j]$ denote the substring beginning at position i and ending at position j. We use the convention that $T[i..j] = \varepsilon$ if $i > j$. The substrings $T[1..j]$ and $T[i..|T|]$ are called prefixes and suffixes of T, respectively. For two strings X and Y, let $\mathsf{lcp}(X, Y)$ denote the length of the longest common prefix of X and Y.

A string X is a *border* of a string W if X is both a prefix and a suffix of W. A border is proper if $X \neq W$. For a non-empty string W, $\mathsf{per}(W)$ denotes its smallest period and $\mathsf{exp}(W) = |W|/\mathsf{per}(W)$ its exponent.

A string W is *closed* if $|W| = 1$, or if there is a non-empty proper border X of W such that X occurs in W only as the prefix and the suffix. An occurrence $T[p..q]$ of a closed string is a *maximal closed substring occurrence*, or *MCS occurrence*, if $p = 1$ or $T[p-1..q]$ is not closed, and $q = |T|$ or $T[p..q+1]$ is not closed [2]. A length-one occurrence is an MCS if and only if it forms a maximal run of length one. Such occurrences can be reported trivially. Hence, in what follows, we consider only MCS occurrences W with $|W| > 1$.

Let W be a closed string of length at least two, and let X be the longest border of W. Then, X occurs in W only as the prefix and the suffix.

For a non-empty string X, let $p_1, \ldots, p_s$ be the list of starting positions of occurrences of X in T arranged in increasing order. A pair (p_t, p_{t+1}) is called a *consecutive occurrence pair* of X for $1 \leq t < s$.

Definition 1 (valid pair). *Let X be a non-empty string and let $i < j$ be two occurrences of X in T. The triple (X, i, j) is a* valid pair *if*

1. *i and j form a consecutive occurrence pair of X;*

2. $i = 1$ *or* $T[i-1] \neq T[j-1]$;
3. $j + |X| - 1 = |T|$ *or* $T[i+|X|] \neq T[j+|X|]$.

The substring spanned by (X, i, j) *is* $\mathsf{span}(X, i, j) = T[i..j+|X|-1]$.

The spanned substring forms an MCS. Moreover, we have:

Lemma 1 (Adapted from [2,5]). *There is a one-to-one correspondence between MCS occurrences of length greater than one and valid pairs, where an MCS occurrence is represented by its longest border and the two occurrences of this border as the prefix and the suffix.*

Definition 2 (horizontal visibility). *Let P be a set of points with distinct x-coordinates, where each point $p \in P$ has a height $h(p)$. Two points $q, r \in P$ with $x(q) < x(r)$ are* visible *if* $\max\{h(p) : x(q) < x(p) < x(r)\} < \min(h(q), h(r))$. *The maximum over the empty set is defined as $-\infty$. For a sequence $h_1, \ldots, h_k$, its horizontal visibility graph is obtained by applying this definition to the points $(1, h_1), \ldots, (k, h_k)$.*

This is the standard horizontal visibility graph introduced by Luque et al. [6]. See Fig. 1 for a concrete example of horizontal visibility graphs.

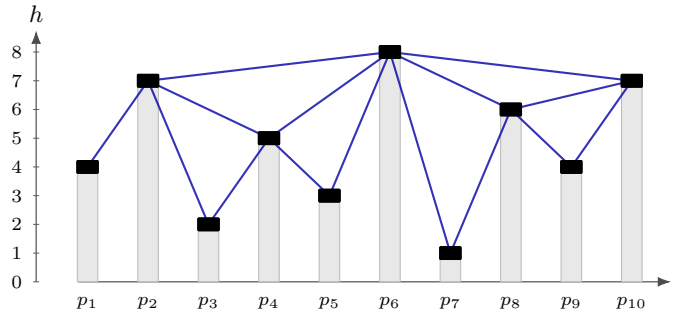


Fig. 1. An example of a horizontal visibility graph. The histogram bar at each position p_i has height h_i; two points are connected when no intermediate bar reaches height $\min\{h_i, h_j\}$. Blue edges are drawn as straight line segments connecting the black endpoint vertices.

We use the following property:

Lemma 2. *The horizontal visibility graph with $k \geq 2$ nodes has at most $2k - 3$ edges.*

Proof. Let the points be $p_1, \ldots, p_k$ in increasing order of their x-coordinates, and draw each visibility edge as an arc above the point sequence. No two arcs cross: if edges (p_i, p_k) and (p_j, p_ℓ) with $i < j < k < \ell$ existed, then

visibility of (p_i, p_k) gives $h(p_j) < \min\{h(p_i), h(p_k)\}$, while visibility of (p_j, p_ℓ) gives $h(p_k) < \min\{h(p_j), h(p_\ell)\}$, a contradiction. Thus the graph is outerplanar.

Extend it, if necessary, to a maximal outerplanar graph on the same k vertices. Let m be its number of edges and let f be the number of bounded faces. All bounded faces are triangles, and the outer face has length k, so counting edge-face incidences gives $2m = k + 3f$. Euler's formula gives $k - m + (f + 1) = 2$. Solving these two equations yields $m = 2k - 3$. The original horizontal visibility graph is a subgraph, and hence has at most $2k - 3$ edges. □

3 Sparse Suffix Tree on RLE

The *run-length encoding* (*RLE*) of a string T is its unique factorization $\mathsf{RLE}(T) = c_1^{e_1} \cdots c_m^{e_m}$, where $e_i \geq 1$ and $c_i \neq c_{i+1}$ for $1 \leq i < m$. Each factor $c_i^{e_i}$ is a *run*, and we write $\rho(T) = m$. Let $S_i = 1 + \sum_{k<i} e_k$ and $E_i = S_i + e_i - 1$ be the starting and ending positions of the ith run, and let $\mathcal{S}_T = \{S_i : 1 \leq i \leq m\}$.

We use the compact trie of the terminal-marked run-start suffixes $\{T[S_i..n]\$: S_i \in \mathcal{S}_T\}$, called the *RLE suffix tree* $\mathsf{RLEST}(T)$. Edge lengths are measured in the expanded string. The terminal character \$ makes all run-start suffixes distinct. Tamakoshi et al. [8] showed that the RLE suffix array and the corresponding RLELCP array for these suffixes can be constructed in $O(m \log m)$ time and $O(m)$ space. From these arrays, $\mathsf{RLEST}(T)$ is obtained in additional $O(m)$ time by the standard suffix-tree construction from a suffix array and an LCP array.

For each leaf ℓ_i corresponding to $T[S_i..|T|]\$$, we store $\mathsf{pos}(\ell_i) = S_i$. If $i > 1$, we also store the character and length of the preceding run, namely $\mathsf{pchar}(\ell_i) = c_{i-1}$ and $\mathsf{plen}(\ell_i) = e_{i-1}$; for $i = 1$ we set $\mathsf{pchar}(\ell_i) = \perp$ and $\mathsf{plen}(\ell_i) = 0$. These values are the truncated RLESA/RLELCP information of Tamakoshi et al. [8].

4 Classification and Family-Size Bounds

Recall that we consider only MCSs W with $|W| \geq 2$. Let (X, i, j) be a valid pair, and let $W = \mathsf{span}(X, i, j)$. When $\mathsf{exp}(W) < 2$, the two border occurrences do not overlap; hence we can write $W = XUX$, where U is the non-empty factor between them. We classify MCS occurrences as follows.

Definition 3 (types). *An MCS occurrence W with longest border X is assigned to one of the following five types:*

Type 1 *:* $\rho(W) = 1$.
Type 2 *:* $\exp(W) < 2$, $\rho(X) = 1$, *and* $\rho(U) = 1$.
Type 3 *:* $\exp(W) < 2$, $\rho(X) = 1$, *and* $\rho(U) \geq 2$.
Type 4 *:* $\exp(W) < 2$ *and* $\rho(X) \geq 2$.
Type 5 *:* $\rho(W) \geq 2$ *and* $\exp(W) \geq 2$.

This section also analyzes the size of the compact representation for each part. The subsequent sections are then devoted only to enumeration.

Types 1 and 5. The MCS occurrences of Types 1 and 5 are exactly maximal repetitions [4]: A substring $R = T[\ell..r]$ with smallest period p is a maximal repetition if $|R| \geq 2p$, and $\ell = 1$ or $T[\ell - 1] \neq T[\ell + p - 1]$, and $r = |T|$ or $T[r + 1] \neq T[r - p + 1]$. Badkobeh et al. [2] observed that MCSs with exponent at least two are maximal repetitions. Thus Types 1 and 5 can be computed by algorithms for computing maximal repetitions on RLE strings. Fujishige et al. [3] showed that all maximal repetitions in an RLE string of size m can be computed in $O(m\alpha(m))$ time and $O(m)$ space, where α denotes the inverse Ackermann function, and that their number is $O(m)$.

Lemma 3. *The MCS occurrences of Type 1 and Type 5 contribute $O(m)$ singleton families, and can be computed in $O(m\alpha(m))$ time and $O(m)$ space.*

Types 2 and 3. For unary-border MCSs, fix a character a. Let $r_1 < \cdots < r_k$ be the a-runs of T and put $h_p = e_{r_p}$. For an a-run r, let $\mathsf{Lout}(r)$ and $\mathsf{Rout}(r)$ be the characters immediately outside the run, using distinct left and right sentinels at the ends of T. For $p < q$, let $M_{p,q} = \max_{p<s<q} h_s$, with $M_{p,q} = 0$ if $q = p + 1$. The two boundary occurrences of a^t at r_p and r_q are consecutive exactly for $M_{p,q} < t \leq \min(h_p, h_q)$. Hence the admissible exponents first form the interval $I_{p,q} = [M_{p,q} + 1, \min(h_p, h_q)]$. In the sense of Definition 2 applied to the height sequence $h_1, \ldots, h_k$, the pair (p, q) is visible exactly when this interval is non-empty.

Lemma 4. *Let $I_{p,q} = [L, R]$ be non-empty. The valid exponents for the pair (r_p, r_q) form an interval J, where $J = I_{p,q}$ or $J = [L, R-1]$. They are represented by the family $\mathcal{U}_a(p, q, J) = \{T[E_{r_p} - t + 1..S_{r_q} + t - 1] : t \in J\}$.*

Proof. The interval condition is precisely the absence of an intermediate a-run of length at least t. If $t < R$, at least one of the two runs is not exhausted on each side, and the two occurrences cannot be extended by the same character. Only $t = R$ can fail maximality: left maximality fails exactly when $h_p = R$ and $\mathsf{Lout}(r_p) = \mathsf{Lout}(r_q)$, and right maximality fails exactly when $h_q = R$ and $\mathsf{Rout}(r_p) = \mathsf{Rout}(r_q)$. Hence, after possibly deleting the last exponent R, the remaining exponents form one interval and are represented by the displayed family. □

Lemma 5. *All Type 2 and Type 3 MCS occurrences can be represented by $O(m)$ unary-border families.*

Proof. For each $a \in \Sigma$, non-empty intervals are exactly the edges of the horizontal visibility graph of $h_1, \ldots, h_k$ from Definition 2, which has $O(k)$ edges by the standard fact stated there. By Lemma 4, each such edge gives at most one unary-border family. Summing over all characters gives $O(m)$ families, since the character classes partition the runs. □

Type 4. For the non-periodic cases with $\rho(X) \geq 2$, singleton families are enough.

Lemma 6. *The number of valid pairs whose longest border has RLE length at least two is $O(m^2)$.*

Proof. Let (X, i, j) be such a valid pair and put $d = j - i$. Since X contains at least one RLE boundary, let b be the leftmost RLE boundary contained in the first occurrence of X. The corresponding position $b + d$ is an RLE boundary in the second occurrence. Charge the pair to the ordered boundary pair $(b, b + d)$.

Fix an ordered boundary pair (b, b'), and let $d = b' - b$. Let L be the maximum number of characters that match immediately to the left of b and b', and let R be the maximum number of characters that match starting at b and b', respectively. Thus the two aligned substrings agree exactly on $T[b-L..b+R-1] = T[b' - L..b' + R - 1]$. If a valid pair is charged to (b, b'), then Conditions 2 and 3 of Definition 1 force $i = b - L$, $|X| = L + R$, and $j = i + d$. Hence (b, b') uniquely determines (X, i, j). There are $O(m^2)$ ordered pairs of RLE boundaries. □

Theorem 2. *There are RLE strings of size m that contain $\Omega(m^2)$ Type 4 MCS occurrences.*

Proof. Let $h \geq 1$. Consider a string consisting of $2h$ blocks $T_h = \prod_{p=1}^{2h} a^{L_p} b^{R_p} \delta_p$, where all δ_p are distinct and different from a, b. For $1 \leq i \leq h$, let $L_i = h - i + 1$ and $R_i = h + i$. For $1 \leq q \leq h$, let $L_{h+q} = h + q$ and $R_{h+q} = q$. For each pair $(i, q) \in [1..h]^2$, let $X_{i,q} = a^{h-i+1} b^q$.

This string occurs at the $a|b$ boundary of block i and at the $a|b$ boundary of block $h + q$. No intermediate block has both enough preceding a's and enough following b's: in the first half, the a-run lengths strictly decrease as the b-run lengths increase, and in the second half the b-run length before block $h + q$ is smaller than q. Thus these two occurrences are consecutive. They are also maximal as a valid pair. At block i the occurrence exhausts the left a-run, whereas at block $h + q$ it does not. On the right, the occurrence exhausts the b-run of block $h + q$, whereas at block i it does not. Hence neither side admits a common extension.

Let $W_{i,q}$ be the substring spanned by these two occurrences. The separator δ_i occurs in $W_{i,q}$ exactly once. If $W_{i,q}$ had a period $p \leq |W_{i,q}|/2$, then shifting this occurrence by one period would give another occurrence with the same adjacent character, a contradiction. Therefore $\mathsf{exp}(W_{i,q}) < 2$. Hence the h^2 pairs yield distinct Type 4 MCS occurrences, and since the RLE size is $\Theta(h)$, the lower bound is $\Omega(m^2)$. □

Combining Lemmas 3, 5, and 6 with Theorem 2, we obtain the following.

Theorem 3. *For any string T of RLE size m, all MCS occurrences of T can be represented by $O(m^2)$ compact families. This bound is worst-case optimal.*

Non-periodic MCSs of Types 2–4 have the form $W = XUX$, where X is the longest border and $U \neq \varepsilon$. Sections 5 and 6 enumerate the unary-border families and the remaining non-unary Type-4 families, respectively. The latter case uses horizontal visibility (Definition 2) on point sets induced by sparse-suffix-tree loci.

5 Unary-Border Enumeration

We now enumerate the Type 2 and Type 3 families described in Lemma 4. The proof of Lemma 5 is constructive: for each character a, it suffices to list the

visible pairs (Definition 2) in the height sequence of the a-runs and output the corresponding interval family after the endpoint maximality test.

Lemma 7. *All unary-border families can be enumerated in $O(m \log \sigma)$ time with $O(m)$ working space, where $\sigma \leq m$ is the alphabet size.*

Proof. Group the runs by their character in $O(m \log \sigma)$ time using a balanced search tree. For each character a, enumerate the horizontal visibility graph of its run-length sequence (Definition 2) by the standard monotone-stack algorithm. This is linear in the number of a-runs. For each visible pair, compute the interval J of Lemma 4 in constant time from the two endpoint run lengths and outside characters, and output the family if J is non-empty. Since the character classes partition the runs, the total work after grouping is $O(m)$ and the working space is $O(m)$. □

6 Non-Periodic MCSs with Non-Unary Borders

Here, we enumerate Type-4 MCS occurrences XUX whose longest border X has $\rho(X) \geq 2$. Write the border as $X = a^K Y$, where a^K is the first RLE factor of X. Then $K \geq 1$, and the remaining core Y is non-empty and starts at a run boundary. This includes the two-run case, where Y is unary.

Let v_0 be the locus of Y in the sparse suffix tree $\mathsf{RLEST}(T)$. A leaf $\ell \in L(v_0)$ represents an occurrence of Y starting at a run boundary. Besides $\mathsf{pchar}(\ell)$ and $\mathsf{plen}(\ell)$, we use $\mathsf{Lout}(\ell)$ for the character immediately before the preceding run, or a sentinel at the left end of T. This value is obtained in constant time from the run represented by ℓ.

Following [2], we use the binarized sparse suffix tree: nodes of outdegree more than two are replaced by binary trees, and all auxiliary nodes inherit the same path-label. In this section, v denotes a node of this binarized tree and $L(v)$ denotes its leaf set. Consider two leaves $\ell, z \in L(v)$ in the two children of a binary node v with $\mathsf{lab}(v) = Y$. If $\mathsf{pchar}(\ell) = \mathsf{pchar}(z) = a \neq \bot$, then they define the candidate border $a^K Y$ with $K = \min(\mathsf{plen}(\ell), \mathsf{plen}(z))$. The candidate is left-maximal exactly when $\mathsf{plen}(\ell) \neq \mathsf{plen}(z)$ or $\mathsf{Lout}(\ell) \neq \mathsf{Lout}(z)$: if the preceding run lengths differ, the shorter run is exhausted while the other occurrence is still preceded by a; if they are equal, both preceding runs are exhausted and the outside characters decide left maximality.

Lemma 8. *Every valid pair whose longest border has RLE length at least two has a unique representation of the form above.*

Proof. Take the longest border X of the valid pair and let a^K be the first RLE factor of X. Write $X = a^K Y$. Then the non-empty core $Y = X[K+1..|X|]$ is uniquely determined and, in both endpoint occurrences of X, the corresponding occurrence of Y starts at a run boundary. Since the two occurrences of X are right-maximal, the corresponding occurrences of Y cannot be followed by the same character. Hence the corresponding leaves of Y belong to distinct original

children of the locus of Y; in particular, this locus is explicit. In the binarized tree, the leaves are separated at a unique lowest binary node with path-label Y. The two occurrences of Y are preceded by a-runs of length at least K, and left maximality implies that one of these runs has length exactly K. If the two preceding-run lengths are equal, validity requires the outside characters to be different. Therefore the construction retains exactly this candidate, and no other choice of the first factor or of the core is possible. □

Visibility reporting. We use the horizontal visibility relation of Definition 2 in this section. Unlike the full suffix-tree algorithm for a fixed border, fixing the core Y does not determine the first-run length K. Thus one query leaf may have several visible partners, corresponding to different values of $K = \min(\mathsf{plen}(\ell), \mathsf{plen}(z))$. The following procedure can be viewed as generating edges of the horizontal visibility graph of an appropriate point set in the sense of Definition 2. We do not construct this graph explicitly; instead, each next visible edge is obtained by a three-sided successor query in $O(\log m)$ time. The following primitive reports these partners output-sensitively. It can be viewed as a local threshold search: the threshold is raised only to the height of a reported point, and a three-sided query jumps over all lower points.

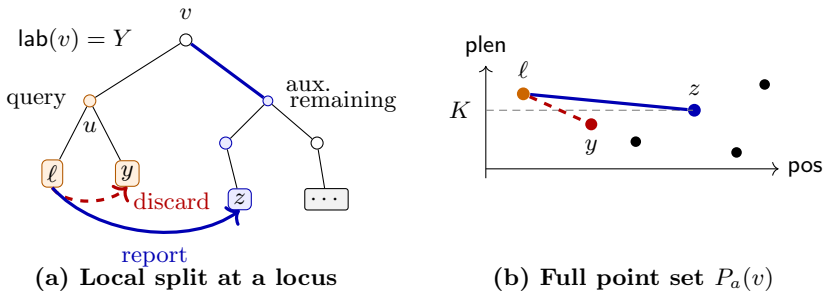
Lemma 9 (three-sided visibility reporting). *Let P be a finite set of points with distinct x-coordinates, where each point $p \in P$ has height $h(p)$. Suppose that P is stored in a structure supporting* $\mathrm{Succ}(s, H) = \arg\min_x\{(x, h) \in P : x > s,\ h > H\}$ *and the symmetric predecessor query in $O(\log |P|)$ time. Then, for any query point $q \in P$, all points of P visible from q can be reported in $O((1 + k_q) \log |P|)$ time, where k_q is the output size.*

Proof. Consider the points to the right of q. Start with $H = -\infty$ and repeatedly ask $\mathrm{Succ}(x(q), H)$. This returns the leftmost point to the right of q whose height is larger than the current threshold H. Report the returned point r; if $h(r) \geq h(q)$, stop, and otherwise set $H \leftarrow h(r)$ and continue. Thus the threshold never scans height values one by one: it jumps to the heights of reported points. Each reported point is visible from q, because all earlier unreported points have height at most the previous value of H. Conversely, any point skipped by this procedure has a reported point of height at least its own height between it and q, and hence is not visible. Thus there is one query per reported point, plus one final query. The left side is symmetric. □

We implement the two queries by McCreight's balanced priority search tree [7]. Let N be larger than every height in P. After replacing each height h by $N - h$, the condition $h > H$ becomes a three-sided range condition $N - h \leq N - H - 1$. Thus successor and predecessor queries are standard MinXInRectangle and MaxXInRectangle queries, respectively, with $O(\log |P|)$ update and query time and linear space.

For a node v of the binarized sparse suffix tree and preceding character a, let

$$P_a(v) = \{(\mathsf{pos}(y), \mathsf{plen}(y), y) : y \in L(v),\ \mathsf{pchar}(y) = a\}.$$



(a) Local split at a locus **(b) Full point set $P_a(v)$**

Fig. 2. Multi-run border enumeration at a locus v. A non-largest child u is used as a query side, while visibility is tested in the full point set $P_a(v)$ stored in $\mathcal{D}_v[a]$. Hence points in the query child, such as y, remain in the point set because they may prevent visibility to an outside leaf, although reports inside $L(u)$ are discarded. A visible outside leaf z gives the candidate border $a^K Y$ with $Y = \mathsf{lab}(v)$ and $K = \min(\mathsf{plen}(\ell), \mathsf{plen}(z))$; in panel (b), the dashed line indicates this height K for the displayed reported pair, not the current threshold used inside the three-sided reporter.

The third component is only a pointer to the leaf; the geometric coordinates are $(\mathsf{pos}(y), \mathsf{plen}(y))$. All visibility queries at v for preceding character a are performed on this point set. This separation is essential: a leaf with a different preceding character represents an occurrence of a different left extension and must not affect the closedness of $a^K\mathsf{lab}(v)$.

The point sets $P_a(v)$ are maintained bottom-up on the binarized sparse suffix tree, and are not rebuilt from scratch. With each processed node x we store a dictionary $\mathcal{D}_x$ indexed by preceding characters; the entry $\mathcal{D}_x[a]$ is a priority search tree storing exactly the points of $P_a(x)$. For a leaf y, $\mathcal{D}_y$ consists of the single point $(\mathsf{pos}(y), \mathsf{plen}(y), y)$ in the entry for $\mathsf{pchar}(y)$, unless $\mathsf{pchar}(y) = \bot$. At an internal binary node, the two child dictionaries are merged by the smaller-to-larger rule to obtain $\mathcal{D}_v$, which is then used for the visibility queries at v.

For two leaves ℓ, z in $P_a(v)$, let K be the smaller of their heights. The two occurrences of $a^K\mathsf{lab}(v)$ are consecutive if and only if no intermediate leaf in $P_a(v)$ has height at least K. Equivalently, the two points are visible. The set $P_a(v)$ is fixed once v and a are fixed; the only changing value during a query is the local threshold used by Lemma 9. Notice that leaves in the same child of v cannot form a right-maximal pair, but they must remain in $P_a(v)$ because such a leaf may be the intermediate leaf that prevents a query leaf and an outside leaf from being visible. Figure 2 illustrates this point-set view together with the same-child discard and outside report.

Lemma 10. *Fix a binary node v of the binarized sparse suffix tree, one child u of v, and a character a. Let q be the number of leaves in $L(u)$ with preceding character a, and let g be the number of visible pairs (ℓ, z) that pass the left-maximality test, where $\ell \in L(u)$, $z \in L(v) \setminus L(u)$, and both leaves have preceding character a. These pairs can be reported in $O((q+g)\log m)$ time.*

Proof. Correctness. Run the visibility reporter on $\mathcal{D}_v[a]$, which stores $P_a(v)$. For each query leaf in $L(u)$, discard reports in the same child and then test left maximality. Visibility gives consecutiveness, the child filter gives right maximality of the core, and the last test gives left maximality. Thus the retained reports are exactly the required candidates.

Discarded reports. Same-child reports are visible pairs inside the q query leaves, so they are contained in a horizontal-visibility graph and contribute $O(q)$ pairs by Lemma 2. A left-maximality failure must have the same height and the same outside character as the query leaf. On either side there is at most one such visible leaf: if such a leaf is visible, then any farther leaf with the same height has this leaf between it and the query leaf, and therefore is not visible. Hence the total number of reports is $O(q + g)$, and each costs $O(\log m)$ time. □

The enumeration procedure is as follows. The binarized sparse suffix tree is processed bottom-up. At a binary node v with children c_1, c_2, let u be a child of minimum leaf-set size, and let w be the other child. After the child dictionaries have been constructed, we form $\mathcal{D}_v$ by keeping $\mathcal{D}_w$ and inserting all points stored in $\mathcal{D}_u$ into the corresponding entries, grouped by their preceding characters. Thus $\mathcal{D}_v[a]$ stores exactly $P_a(v)$ for every character a. This merging is performed also at nodes with empty path-labels, because the dictionary of v must be passed to its parent.

If $Y = \mathsf{lab}(v)$ is non-empty, then each leaf $\ell \in L(u)$ with $a = \mathsf{pchar}(\ell) \neq \bot$ is used as a query leaf. We run the visibility reporting algorithm of Lemma 9 on $\mathcal{D}_v[a]$, which represents the whole point set $P_a(v)$. A reported leaf z is ignored if $z \in L(u)$. Otherwise, if $\mathsf{plen}(\ell) \neq \mathsf{plen}(z)$ or $\mathsf{Lout}(\ell) \neq \mathsf{Lout}(z)$, we set $K = \min(\mathsf{plen}(\ell), \mathsf{plen}(z))$, $i = \min(\mathsf{pos}(\ell), \mathsf{pos}(z)) - K$, and $j = \max(\mathsf{pos}(\ell), \mathsf{pos}(z)) - K$, and output the candidate only when $|a^K Y| < j - i$. This final test removes exactly the periodic candidates. Since every leaf pair is separated at a unique lowest binary node and only one side is queried there, no candidate is generated twice.

Here, LeftMax(ℓ, z) denotes the test $\mathsf{plen}(\ell) \neq \mathsf{plen}(z)$ or $\mathsf{Lout}(\ell) \neq \mathsf{Lout}(z)$. The loop in Line 11 is implemented by the visibility reporting of Lemma 9: each next visible leaf is obtained by a three-sided successor or predecessor query on $\mathcal{D}_v[a]$. The output object Multi(v, ℓ, z, K) stores the singleton candidate determined by the two starting positions computed in Lines 16–17. The condition in Line 18 is the non-periodicity test.

Lemma 11. *The above enumeration outputs exactly the valid pairs whose longest border has RLE length at least two and whose spanned MCS has exponent smaller than two, without duplication.*

Proof. Soundness. An output pair uses two leaves in the two children of a binary node with path-label $Y \neq \varepsilon$. By the construction of the binarized tree, the leaves belong to distinct original children of the locus of Y, and hence the two core occurrences are right-maximal. Visibility excludes an intermediate occurrence of $a^K Y$, and the explicit test on plen and Lout gives left maximality. Therefore the

Algorithm 1: Non-unary enumeration at a binary node v.

Input: A binary node v whose children have already been processed.
Output: The dictionary $\mathcal{D}_v$ and the output families generated at v.

```
1  procedure ProcessNode(v):
2      let c_1, c_2 be the children of v
3      let u be a child of v with minimum leaf-set size, and let w be the other child
4      form D_v by keeping D_w and inserting all points of D_u into it, grouped by
         pchar
5      Y ← lab(v)
6      if Y = ε then
7          return D_v
8      foreach leaf ℓ ∈ L(u) do
9          a ← pchar(ℓ)
10         if a ≠ ⊥ then
11             foreach leaf z visible from ℓ in D_v[a] do
12                 if z ∈ L(u) then
13                     continue
14                 if LeftMax(ℓ, z) then
15                     K ← min(plen(ℓ), plen(z))
16                     i ← min(pos(ℓ), pos(z)) − K
17                     j ← max(pos(ℓ), pos(z)) − K
18                     if |a^K Y| < j − i then
19                         output Multi(v, ℓ, z, K)
20     return D_v
```

pair is valid. Since Y is non-empty, the border has at least two RLE factors, and the final length test removes exactly the periodic cases.

Completeness and uniqueness. Take a valid non-periodic pair of this type. By Lemma 8, its longest border is represented uniquely as $X = a^K Y$ with non-empty Y. In the binarized tree, the two corresponding leaves are separated at a unique lowest binary node with path-label Y. At this node, the smaller child is queried, so the pair is reported, passes the left-maximality test, and passes the non-periodicity test. No other binary node separates the same two leaves. □

Lemma 12. *The non-unary Type-4 families can be enumerated in* $O(m \log^2 m + f \log m)$ *time, where* f *is their number.*

Proof. Queries. A leaf is queried only when it belongs to the smaller child of a binary node. Whenever this happens, the parent subtree has at least twice as many leaves as that child. Hence a fixed leaf is queried $O(\log m)$ times, and there are $O(m \log m)$ query leaves in total.

Structures. The dictionaries are maintained bottom-up on the binarized sparse suffix tree. At a binary node v, we keep the dictionary of the larger child and insert all points of the smaller child into it, grouped by their preceding characters. If an entry for a character does not exist in the larger dictionary, the corresponding priority search tree of the smaller child is moved as a whole. Otherwise its points

are inserted one by one. Whenever a point is inserted into an existing structure, the size of the subtree containing it at least doubles. Hence each point is inserted $O(\log m)$ times. Since each insertion and dictionary lookup costs $O(\log m)$ time, the total structure-maintenance time is $O(m \log^2 m)$. At any moment each leaf is stored in exactly one live structure, so the working space is $O(m)$.

Reports. In Lemma 10, the q terms sum to the number of query leaves. The g terms count reports passing the left-maximality test before the non-periodicity test. Each candidate pair is generated only at its unique separating binary node and only from the smaller side. The kept non-periodic candidates are exactly the output families counted by f. The kept periodic candidates are Type-5 MCSs; by the unique core representation, they are charged once, and there are $O(m)$ of them. Thus reporting costs $O((f + m) \log m)$, and the claimed bound follows. □

References

1. Akagi, T., Okabe, K., Mieno, T., Nakashima, Y., Inenaga, S.: Minimal absent words on run-length encoded strings. In: Proceedings of the 33rd Annual Symposium on Combinatorial Pattern Matching (CPM 2022). pp. 27:1–27:17 (2022)
2. Badkobeh, G., Luca, A.D., Fici, G., Puglisi, S.J.: Finding maximal closed substrings. Theor. Comput. Sci. **1060**, 115628 (2026)
3. Fujishige, Y., Nakashima, Y., Inenaga, S., Bannai, H., Takeda, M.: Almost linear time computation of maximal repetitions in run length encoded strings. In: ISAAC 2017. pp. 33:1–33:12 (2017)
4. Kolpakov, R.M., Kucherov, G.: Finding maximal repetitions in a word in linear time. In: FOCS 1999. pp. 596–604 (1999)
5. Kosolobov, D.: Closed repeats. CoRR **abs/2410.00209** (2024)
6. Luque, B., Lacasa, L., Ballesteros, F., Luque, J.: Horizontal visibility graphs: Exact results for random time series. Physical Review E **80**(4), 046103 (2009). `https://doi.org/10.1103/PhysRevE.80.046103`
7. McCreight, E.M.: Priority search trees. SIAM Journal on Computing **14**(2), 257–276 (1985)
8. Tamakoshi, Y., Goto, K., Inenaga, S., Bannai, H., Takeda, M.: An opportunistic text indexing structure based on run length encoding. In: CIAC 2015. pp. 390–402 (2015)